WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Face à KAPE et EZ
WAC (Windows Artefacts Collector)

Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

WAC — Windows Artefacts Collector

Face à KAPE et aux outils d'Eric Zimmerman

Version du logiciel	1.4.0
Édition du document	octobre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC)
	École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document confronte WAC aux outils de référence de la communauté : KAPE pour la collecte, les outils en ligne de commande d'Eric Zimmerman pour le décodage. Sur les mêmes pièces d'une même machine, il compare les temps d'exécution, la justesse des valeurs — chaque écart étant tranché par une référence indépendante — et les formats de sortie. Il dit ce que la confrontation a appris à WAC, et ce qu'elle lui reproche encore.

Table des matières

1 La démarche **2**

1.1 Ce qui est comparé 2

1.2 Qui a raison quand ils diffèrent 2

1.3 Le banc d'essai 2

2 Synthèse **4**

3 Temps d'exécution **6**

3.1 Décodage 6

3.2 Collecte 6

4 Précision **8**

4.1 En faveur de WAC 8

4.2 En faveur des outils EZ 9

5 Formats de sortie **10**

5.1 Ce que produit chaque outil 10

5.2 Ce qui les distingue 10

6 Ce que la confrontation a appris à WAC **12**

7 Limites de la comparaison **13**

CHAPITRE 1

La démarche

1.1 Ce qui est comparé

Deux questions, deux comparaisons.

Collecte : WAC face à KAPE
Les deux outils collectent la même machine l'un après l'autre ; on compare ce que chacun copie, le temps qu'il y met, et l'empreinte SHA-1 des fichiers qu'ils copient tous deux.

Décodage : WAC face aux outils EZ
Sur les *mêmes pièces* — le magasin de pièces d'une collecte WAC —, WAC convertit un artefact et l'outil d'Eric Zimmerman correspondant le décode ; les deux sorties sont comparées enregistrement par enregistrement et champ par champ.

1.2 Qui a raison quand ils diffèrent

Un écart entre deux outils ne dit pas lequel se trompe. Chaque écart a donc été tranché par une troisième source, indépendante des deux :

- Windows lui-même : le XML qu'il rend pour un événement (`Get-WinEvent`), la date d'un fichier, l'identifiant d'objet NTFS (`fsutil objectid`), le registre en ligne, l'empreinte d'un fichier (`Get-FileHash`) ;
- une autre implémentation : `dissect.esedb` (Fox-IT) pour les bases ESE ;
- le format lui-même : numéros de séquence et sommes Marvin32 des journaux de ruche, lus octet par octet ;
- le recoupement de deux artefacts de la même machine : le Prefetch pour l'exécution, l'horodatage d'une ligne pour ses autres dates.

Chaque écart est ensuite classé : *défaut de WAC* (corrigé), *erreur de l'outil EZ* (vérifiée), ou *convention* de l'un des deux — deux formes d'une même valeur.

1.3 Le banc d'essai

Machine	Windows 11 Professionnel (build 26200), machine virtuelle
Outils EZ	version 2026.5.0 (.NET 9.0.20)
KAPE	version 1.3.0.2, cible `KapeTriage`
WAC	version 1.4.0 et ses corrections issues de cette confrontation

Tout se rejoue : `vmtest/ez-compare.sh <artefact>` chronomètre WAC et l'outil EZ sur les mêmes pièces puis compare leurs sorties ; `vmtest/kape-compare.sh` fait de même pour la collecte. Les outils EZ ont été lancés avec leur pleine précision de date (`--mp`, ou `--dt` au format à sept décimales quand `--mp` ne suffit pas), pour ne pas leur imputer une perte de précision qu'un réglage évite.

CHAPITRE 2

Synthèse

Artefact	Volume	Résultat	Erreurs de l'outil EZ, vérifiées
$MFT — MFTECmd	362 081 enreg.	chemins, états, 8 dates identiques	tailles de 1 490 fichiers WOF et 3 journaux ouverts : Windows donne celles de WAC
EVTX — EvtxECmd	190 613 évén.	champs système identiques ; valeurs expliquées	6 valeurs fausses (Windows) ; 335 événements Perflib illisibles ; journaux vidés ignorés
Prefetch — PECmd	379 fichiers	tout identique	empreinte privée de son zéro de tête (19)
Raccourcis — LECmd	3 fichiers	identiques au 100 ns	—
Jump lists — JLECmd	13 raccourcis	identiques au 100 ns	—
Shellbags — SBECmd	27 dossiers	identiques	NodeSlot faux au premier niveau (7) ; HasExplored toujours faux
Amcache — AmcacheParser	1 258 entrées	124 colonnes identiques	2 340 valeurs inventées ; SHA-1 partiel non signalé
ShimCache — AppCompat-CacheParser	232 entrées	identiques	indicateur d'exécution trompeur
Corbeille — RBCmd	19 fichiers	identiques	—
SRUM — SrumECmd	60 804 lignes	identiques	17 lignes d'énergie fausses (dissect)
Registre — RECmd	9 artefacts	identiques	4 ruches ramenées à un état ancien
Collecte — KAPE	1 173 fichiers communs	SHA-1 identiques	—

> **À retenir**
>
> Sur tout ce que les deux côtés lisent, WAC donne les mêmes valeurs que les outils de référence, ou des valeurs plus justes, établies à chaque fois par une source indépendante. La confrontation a aussi révélé 23 défauts de WAC, tous corrigés (chapitre 6), et des lacunes de couverture, ouvertes au suivi (section 4.2).

CHAPITRE 3

Temps d’exécution

3.1 Décodage

Même machine, mêmes pièces ; chaque outil ne traite que son artefact.

Artefact	WAC	Outil EZ	Ce que fait WAC en plus
$MFT (380 Mo)	7,3 s	8,1 s	deux volumes (C : et G :) contre un seul
EVTX (305 journaux)	16,8 s	22,0 s	message en clair reconstitué pour chaque événement
Registre (lot Kroll)	2,0 s	7,6 à 13,6 s	toutes les ruches rejouées, tous les artefacts du registre
SRUM (21 Mo)	3,2 s	2,5 s	2 tables de plus, identités résolues
Prefetch (380)	3,6 s	0,7 s	heure locale datée, chemins avec lettre de volume
Raccourcis, jump lists	1,8 s	0,3 s	
Corbeille	1,9 s	0,5 s	
Amcache, ShimCache, shellbags	2,0 s	0,5 à 1,4 s	un seul passage pour les trois (clé registry)

Sur les gros artefacts, WAC est plus rapide. Sur les petits, les outils EZ le sont : toute conversion WAC paie un coût fixe de 1,5 à 3,4 s, mesuré avec un artefact trivial. Ce coût est celui de la copie de travail vérifiée des ruches, du rejeu de leurs journaux, et de la lecture du fuseau et des profils du suspect. C’est ce qui permet à WAC de dater chaque heure locale avec le décalage en vigueur à sa date, et de donner la lettre de volume ; les outils EZ n’en donnent pas.

3.2 Collecte

Outil, réglage	Durée	Pièces	Volume
KAPE, `KapeTriage`	58 s	1 047	0,97 Go
WAC, kit, sans binaires	47 s	1 898	2,54 Go

Outil, réglage	Durée	Pièces	Volume
WAC, kit, sans binaires, avant B-033	212 s	2 622	1,69 Go
WAC, kit complet (`binary: unverified`), avant B-033	454 s	61 294	3,6 Go

Les deux premières lignes sont mesurées dans la même exécution, l'une après l'autre. À périmètre comparable, WAC collecte désormais plus vite que KAPE, en ramenant plus de deux fois plus de données. Il était cinq fois plus lent au départ (fiche B-033) :

— chaque lecture brute rouvrait son volume et reparcourait ses chemins depuis la racine ; les volumes sont maintenant ouverts une fois, avec le cache de leurs répertoires ;

— la collecte prenait les fichiers des 1 426 fournisseurs d'événements déclarés (739 Mo, 61 s) ; elle ne prend plus que ceux des fournisseurs que citent les journaux collectés (324 sur la VM, 178 Mo, 15 s), ce qui suffit à reconstituer le texte de chaque événement.

Le kit complet vérifie en outre la signature Authenticode de chaque exécutable de la machine et copie ceux dont la signature ne tient pas, ce que KAPE ne fait pas.

CHAPITRE 4

Précision

4.1 En faveur de WAC

Journaux de ruche rejoués dans l'ordre
Les journaux `.LOG1` et `.LOG2` gardent, derrière les entrées récentes, les restes d'une génération précédente. RECmd les applique dans l'ordre du fichier, donc après les plus récentes. Il ramène ainsi SYSTEM, SECURITY, DEFAULT et une ruche de profil de service à un état plus ancien : dans SYSTEM, la séquence 0x3618 au lieu de 0x3620, et quatre dernières exécutions BAM affichées 13 minutes trop tôt. WAC trie les entrées par séquence et s'arrête à la rupture de chaîne. Les journaux eux-mêmes le vérifient, sommes Marvin32 comprises.

Événements des journaux vidés
Un journal vidé garde ses enregistrements : WAC les relit et les marque `EvtRecovered`, sans jamais les présenter comme vivants ; EvtxECmd ne les voit pas.

Valeurs d'événements comme Windows les rend
Hexadécimal `0x3e7`, GUID entre accolades, booléens `true`, binaire `69060000`, dates avec leur `Z`, marques U+200E conservées : les formes qu'affiche l'Observateur d'événements et que l'on recherche. EvtxECmd en change huit, et se trompe sur six valeurs, par exemple un booléen `False` que Windows rend `true`.

Rien d'inventé
Une valeur absente de la source n'est pas émise. AmcacheParser écrit 2 340 valeurs par défaut absentes de la ruche (`IsPeFile: False` pour un exécutable PE), SrumECmd 7 086 (0 ou -1).

SHA-1 partiel de l'Amcache signalé
Pour un fichier de plus de 30 Mio, Amcache ne hache que les 31 457 280 premiers octets (mesuré sur `mrt.exe`). WAC le dit (`Sha1CoversBytes`) ; sans cela, la comparaison avec le SHA-1 du fichier entier ferait conclure à tort à une modification.

Pas de champ trompeur
L'indicateur « exécuté » du ShimCache, que donne AppCompatCacheParser, dit « non » pour 105 des 117 binaires dont le Prefetch prouve l'exécution. WAC ne le relève pas : ce « non » ferait conclure qu'un programme n'a jamais tourné.

Précision au 100 ns
WAC donne chaque date à la précision de sa source. Par défaut, PECmd, LECmd et SrumECmd arrondissent à la seconde, et LECmd ignore `--mp` dans son CSV.

Heure locale datée
Chaque date est donnée en UTC et en heure locale du suspect, avec le décalage en vigueur

à cette date. Les outils EZ ne donnent que l'UTC, sans le signaler par un `Z`.

Lectures d'autrui vérifiées
Les « FirstInteracted » et « LastInteracted » de SBECmd, le « CreatedOn » des comptes chez RECmd sont des lectures de dates d'écriture de clés. WAC publie ces dates elles-mêmes : chacun en tire la lecture qu'il veut, sans en hériter une toute faite. SBECmd donne en outre un `NodeSlot` faux (0) aux dossiers de premier niveau.

Erreurs de lecture ESE de SrumECmd
17 lignes d'énergie sur 887 portent le contenu d'une autre ligne ou un identifiant absent de la table ; dissect.esedb donne raison à WAC.

Collecte
Chaque pièce de WAC porte MD5, SHA-1 et SHA-256, les huit dates NTFS de son original et son entrée MFT, dans un manifeste scellé. KAPE donne un SHA-1 et trois dates.

4.2 En faveur des outils EZ

Couverture du registre
Le lot Kroll de RECmd lit des clés que WAC ignore : programmes installés, exclusions de Defender, RecentDocs, MountPoints2, réseaux connus, état de la mise à jour des dates d'accès NTFS, heure d'arrêt, entre autres. Fiche B-031.

Couverture de fichiers
KapeTriage ne copie plus, face à WAC, que les navigateurs et la chronologie Windows, prévus (B-016, B-010), et les `desktop.ini`, écartés. Les raccourcis hors de Recent, la ruche du profil Default, `$Secure:$SDS`, `$Boot`, `$UsnJrnl:$Max`, `$TxfLog`, les traces WPP de Defender et les clés maîtresses DPAPI, qu'il était seul à prendre, sont collectés et convertis depuis la fiche B-032.

Vitesse sur les petits artefacts et en collecte
Voir le chapitre précédent.

Interprétations prêtes à lire
Les outils EZ découpent et nomment certains champs : exécutable, date et description d'une application SRUM, noms des dossiers connus, extension d'un fichier. WAC donne la valeur brute, telle que Windows la stocke.

> **Limite**
> Ce que WAC ne collecte pas, il ne peut pas le mettre en balance : la couverture de KAPE reste plus large sur les navigateurs et plusieurs clés de registre, jusqu'à ce que B-016, B-031 et B-032 soient faits.

CHAPITRE 5

Formats de sortie

5.1 Ce que produit chaque outil

Outil	Formats	Remarque
AmcacheParser	CSV	8 fichiers, un par table
AppCompatCacheParser	CSV	
RBCmd	CSV	
SrumECmd	CSV	7 fichiers
EvtxECmd	CSV, JSON, XML	
JLECmd	CSV, JSON, HTML	
LECmd	CSV, JSON, HTML, XML	
MFTECmd	CSV, JSON, bodyfile	
PECmd	CSV, JSON, HTML	
RECmd	CSV, JSON	plus un CSV par greffon
SBECmd	CSV, JSON	
KAPE	copie des fichiers, journal de copie CSV	
WAC	JSON pour chaque artefact	un fichier par artefact, plus le manifeste scellé

5.2 Ce qui les distingue

Quatre outils sur onze ne produisent que du CSV. Chez les autres, le JSON est la transcription de la ligne CSV : une ligne de texte par enregistrement (JSON Lines), à plat.

Types
Chez les outils EZ, les nombres sont des chaînes : un compteur d'exécutions s'écrit `"RunCount":"458"`, un numéro d'enregistrement `"EventRecordId":"21572"`. WAC écrit des nombres, des booléens, des chaînes.

Structures
Chez les outils EZ, une liste devient une chaîne jointe par des virgules (les fichiers chargés d'un Prefetch, ses répertoires), et les données d'un événement sont un JSON échappé *dans* une chaîne (`Payload`), à décoder une seconde fois. WAC imbrique : les volumes d'un Prefetch

et leurs répertoires, les éléments d'un raccourci, les paires `Name`/`Value` d'un événement.

Dates
Chez les outils EZ, l'UTC n'est pas signalé (`2026-09-30 23:04:46`) et la précision dépend d'une option. WAC donne l'ISO 8601 complet, en UTC (…`Z`) et en heure locale avec son décalage (…`+02:00`).

Absence de valeur
Les outils EZ écrivent une valeur par défaut, ou une cellule vide. WAC n'émet pas le champ ; un artefact non collecté est déclaré tel (`NotRequested`, `NotCollected`), pour qu'une liste vide ne passe pas pour une absence de trace.

Provenance
Le manifeste de WAC relie chaque valeur à la pièce dont elle vient, avec ses empreintes et ses dates d'origine.

Console
Les outils EZ détaillent leur travail sur la console ; certains y écrivent le décodage lui-même quand aucun fichier n'est demandé. WAC n'y écrit que l'avancement : tout ce qu'il trouve est dans les fichiers.

> **À retenir**
> Pour un analyste, la différence se voit au premier traitement : un JSON de WAC se charge tel quel dans un outil d'analyse ou dans WAE, sans rien redécouper ; une sortie EZ se traite comme un tableur, dont il faut reconvertir les nombres, redécouper les listes et redécoder les données d'événement.

CHAPITRE 6

Ce que la confrontation a appris à WAC

Chaque défaut ci-dessous produisait un JSON valide et faux, ou incomplet. Tous sont corrigés, avec un contrôle qui les aurait pris.

Artefact	Défaut corrigé
$MFT	enregistrements d'extension périmés fusionnés avec la base
EVTX	flottants tronqués à 6 chiffres ; binaire en « 69 06 00 00 » ; entiers hexadécimaux et GUID au mauvais format ; ProcessingErrorData ignoré (21 événements sans données) ; événements des journaux vidés non lus
Raccourcis	URI collée au bloc qui la suit ; lecture décalée de 2 octets sans liste d'ID ; blocs ExtraData non lus (machine, suivi) ; taille de la cible non lue ; DROID nuls émis
Jump lists	nombre d'ouvertures et rang MRU non lus ; format de DestList version 1 non pris en charge ; nom d'application altéré
Shellbags	dossier perdu avec sa descendance ; date de la clé du dossier, rang et NodeSlot absents ; dossier « Accueil » sans nom
Amcache	2 tables sur 14 et 4 à 6 valeurs lues : ni SHA-1, ni taille, ni pilote, ni périphérique
ShimCache	tabulations des applications empaquetées remplacées par des espaces
SRUM	dates FILETIME laissées en nombres ; texte UTF-16 lu comme de l'ANSI
Registre	BAM limité aux comptes ayant un profil (un compte supprimé perdait ses exécutions) ; durée de premier plan de UserAssist non lue
Harnais	un contrôle qui plantait interrompait tous les suivants sans échec visible

CHAPITRE 7

Limites de la comparaison

— Une seule machine, virtuelle, sous Windows 11 : certains artefacts y sont peu nombreux (3 raccourcis de Recent, 2 clés USB). Les formats sont les mêmes sur une machine réelle, mais les cas rares n'y sont pas tous représentés.

— Les deux collectes se suivent au lieu d'être simultanées : les fichiers écrits entre elles ($MFT, journaux, Prefetch) diffèrent légitimement.

— Les outils EZ ont été lancés avec leurs options de précision maximale ; par défaut, ils perdent davantage.

— La comparaison porte sur les artefacts communs. Ce que seul l'un des outils lit n'a pas de référence ici : son exactitude repose sur les contrôles du dossier de validation.